

HelixPlay

HelixPlay

Verwandeln Sie jedes GPU-Gerät in Ihr eigenes Cloud-Gaming-System.

Zusammenfassung

HelixPlay ist eine selbst gehostete Cloud-Gaming-Plattform, die jedes mit GPU ausgestattete Gerät in einen Remote-Streaming-Host verwandelt und Konsolen-Qualität auf Desktop-, Mobil-, TV- und Browser-Clients überträgt. Die Plattform ist als Go-zentrisches Monorepo mit 46 Submodulen und einem Triple-Stack-Client aufgebaut und kann für Partner weißgelabelt werden.

Kurzbeschreibung

HelixPlay ist eine selbst gehostete, offene und weißgelabelte Cloud-Gaming-Plattform. Sie verwandelt jedes GPU-Gerät in einen Remote-Streaming-Host und überträgt Konsolen-Qualität auf Desktop-, Mobil-, TV- und Browser-Clients über WebRTC/QUIC, mit einem Go-Kern und einem Wails/Flutter/Angular-Client-Stack.

Ausführliche Beschreibung

HelixPlay ist eine Cloud-Gaming-Plattform, die als Go-zentrisches Monorepo mit 46 Git-Submodulen entwickelt wurde. Sie verwandelt jeden bereits vorhandenen Gaming-PC in einen Streaming-Host und liefert Konsolen-Qualität an Desktop-, Mobil-, TV- und Browser-Clients – selbst gehostet, offen und für Partner weißgelabelt. Die Botschaft ist klar: Ihre Hardware, Ihr Service, Ihre Marke, ohne Cloud-Drittanbieter dazwischen.

Ihr prägendes Designmerkmal ist die Triple-Stack-Client-Konvergenz – eine architektonische Wette, die sich an allen anderen Stellen auszahlt. Eine Wails-Desktop-App, eine Flutter-Mobil-/TV-App und ein Angular-Webclient basieren auf einem

einzigem Go-Kern, der für den Browser zu WASM kompiliert wird. Dadurch wird das Verhalten einmal geschrieben und auf allen Oberflächen geteilt, statt dreifach verzweigt zu werden. Darunter liegt der Echtzeit-Medienpfad: Erfassung → Kodierung → Paketierung → Übertragung → Dekodierung → Rendering, verbunden mit plattformspezifischer Erfassung (DXGI / ScreenCaptureKit / PipeWire) und Hardware-Encodern (NVENC / QSV / AMF / VideoToolbox), sodass die GPU die Hauptlast trägt. Transportiert wird über WebRTC (Pion v4), QUIC (quic-go) und benutzerdefinierte UDP-Datagramme, die auf Latenz statt auf Bequemlichkeit optimiert sind. Der Backend-Kern verwaltet Sitzungen, Mandanten, Kataloge und Authentifizierung; ein Host-Agent übernimmt Erfassung, Kodierung und Transport am Edge; und mDNS/Rendezvous verbindet die Erkennung, sodass Clients ihren Host ohne manuelle Konfiguration finden.

HelixPlay ist von Grund auf für White-Label-SaaS konzipiert – mit mandantenspezifischem Theming, Katalogfilterung, OAuth2 und Abrechnung –, sodass Partner einen vollständig gebrandeten Service aufbauen können, statt nur eine oberflächliche Anpassung vorzunehmen. Zudem ist die Plattform durchgängig containerbasiert: Jeder Dienst, jede Datenbank, jeder Build, jeder Test und jede Prüfung läuft in Containern, was die gesamte Plattform reproduzierbar macht – sowohl für die Bereitstellung als auch für die Verifizierung. Wie der Rest der Helix-Familie folgt sie einer „Anti-Bluff-Verfassung“, bei der ein grüner Test echte, für Endnutzer brauchbare Funktionalität garantieren soll – und nicht nur ein erfolgreiches Mocking.

Warum wir es entwickelt haben

Kommerzielles Cloud-Gaming ist geschlossen, zentralisiert und gemietet. HelixPlay wurde entwickelt, damit jeder mit einem GPU-Gerät seinen eigenen Streaming-Host betreiben kann – offen, selbst gehostet und weißgelabelt – statt von einem Drittanbieter abhängig zu sein.

Warum es ein Game-Changer ist

Es vereint drei Dinge, die kommerzielle Dienste getrennt halten: Self-Hosting auf Hardware, die Sie kontrollieren, einen gemeinsamen **Go**-Kern, der drei Client-Stacks antreibt, sodass Funktionen überall gleichzeitig landen, und White-Label-Multi-Tenancy. Das Ergebnis: Ein Partner kann einen vollständig gebrandeten Cloud-Gaming-Dienst auf *eigenen* GPUs starten – und besitzt damit das Erlebnis, die Nutzer und die Wirtschaftlichkeit –, statt Kapazitäten in der Cloud eines anderen zu vertreiben und sich dessen Grenzen zu unterwerfen.

Was innovativ ist

- **Konvergenz der drei Client-Stacks** – Wails, **Flutter** und **Angular** laufen auf einem einzigen **Go**-Kern (WASM im Browser), sodass Desktop, Mobile, TV und Web eine einzige Implementierung teilen, statt drei auseinanderlaufende.
- **Self-hostbares, White-Label-SaaS** – pro-Tenant-Theming, Katalogfilterung, OAuth2 und Abrechnung sind integriert, sodass die Plattform als brandbares Produkt ausgeliefert wird, nicht als Demo.
- **Moderner Low-Latency-Transport** – WebRTC (Pion), **QUIC** und ein maßgeschneidertes **UDP**, kombiniert mit plattformspezifischer Hardware-Encoder-Auswahl (NVENC / QSV / AMF / VideoToolbox), optimiert für Reaktionsgeschwindigkeit statt Bequemlichkeit.
- **46 entkoppelte Module** – sauber getrennte Komponenten mit container-nativem Ansatz: Jeder Dienst, jede Datenbank, jeder Build, Test und Scan läuft in einem Container.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Low-Latency-Streaming über heterogene Hardware.** Jedes Betriebssystem und jede **GPU**-Implementierung bietet Capture und Encoding unterschiedlich an, und Latenz verzeiht keine Fehler. Gelöst durch einen plattformbewussten Capture/Encode-Pfad – DXGI / ScreenCaptureKit / PipeWire speisen NVENC / QSV / AMF / VideoToolbox –, übertragen über WebRTC / **QUIC** / **UDP**, sodass jedes Gerät seinen eigenen schnellsten nativen Weg zu den Pixeln nutzt.
- **Ein Produkt für Desktop, Mobile, TV und Web.** Gelöst durch drei Client-Stacks (Wails, **Flutter**, **Angular**), die einen einzigen **Go**-Kern teilen, der für den Browser zu WASM kompiliert wird. So erscheint eine einmal geschriebene Korrektur oder Funktion auf allen vier Oberflächen, statt viermal portiert zu werden.
- **Multi-Tenant-White-Label-Betrieb.** Gelöst durch die direkte Integration von pro-Tenant-Theming, Katalogfilterung, OAuth2 und Abrechnung in den Backend-Kern, sodass Tenant-Isolation und Branding Plattform-Grundfunktionen sind und nicht pro Kunde geforkt werden müssen.

Tech-Stack

- **Go (1.26.2 Root / 1.25+ Submodule)** – das gemeinsame Backend und Host-Agent; eine Sprache, die sowohl zu nativen Binaries als auch zu WASM kompiliert wird – das ermöglicht das Single-Core-, Multi-Client-Design.

- **Wails v2** – der Desktop-Client, der den **Go**-Kern an eine eingebettete Webview bindet, sodass die Desktop-App die Kernlogik direkt wiederverwendet, statt sie neu zu implementieren.
- **Flutter 3.29+** – der Mobile/TV-Client, der über FFI auf den **Go**-Kern zugreift, für eine native UI auf Smartphones und Fernsehern – ohne zweites Backend.
- **Angular 17+** – der Web-Client, der denselben **Go**-Kern als WASM ausführt, sodass der Browser eine vollwertige Oberfläche ist, keine abgespeckte Version.
- **WebRTC / Pion v4, QUIC / quic-go, maßgeschneidertes UDP** – drei Echtzeit-Transportprotokolle, damit die Plattform für jedes Netzwerk und jeden Client den Pfad mit der geringsten Latenz wählen kann.
- **Hardware-Encoder (NVENC / QSV / AMF / VideoToolbox) und Plattform-Capture (DXGI / ScreenCaptureKit / PipeWire)** – der **GPU**-beschleunigte Capture- und Encode-Pfad, plattformspezifisch ausgewählt, damit das Encoding nie zum CPU-Flaschenhals wird.
- **Container (Docker/Podman)** – jeder Dienst, jede Datenbank, jeder Build, Test und Scan läuft containerisiert, sodass das gesamte System reproduzierbar bereitgestellt und überprüft werden kann.
- **mDNS / Rendezvous** – Zero-Config-Host-Erkennung, damit Clients ihren Streaming-Host automatisch im Netzwerk finden.

Status- und Ehrlichkeitshinweise

- **Status: Beta.** Die im README genannten Latenzziele (≤ 30 ms LAN / ≤ 50 ms WAN p999), die Einordnung als „Konsolenklasse / PS4-Pro-Klasse“ sowie die Anzahl der Testmatrix-Zellen sind vom Projekt selbst definierte Designziele, nicht unabhängig validiert, und werden als solche dargestellt.
- **Lizenz: Noch nicht festgelegt.** Über die GitHub API wurde keine LIZENZ erkannt — UNBESTÄTIGT / nicht deklariert.

Prioritätsstufe: Helix-primär.